

Relatório - Lista 3

Equipe: Angel Jhesua Machado Rodriguez e João Guilherme de Souza Gomes

Disciplina: Estrutura de Dados

Professor: Matheus Telles Werner

Mai de 2026

Implementação dos algoritmos de ordenação

O ponto de contato entre os dois algoritmos é a função `comesFirst`. Ela compara dois jogadores `a` e `b` e determina qual deve vir antes na fila. O critério segue exatamente o enunciado: primeiro o de menor score e, em caso de empate, o de menor timestamp. Essa função centraliza a regra de comparação e evita duplicação de lógica nas implementações de *insertion sort* e *merge sort*.

Insertion Sort

O `sortByScoreInsertion` foi implementado de forma direta. A ideia foi considerar que o elemento inicial do array já está ordenado e inserir, a cada passo, o próximo elemento na posição correta. Para isso, o algoritmo percorre o vetor a partir da segunda posição, salva o elemento atual em `current` e o compara com os elementos anteriores da parte ordenada.

Enquanto `current` deve vir antes do elemento analisado, é deslocado em uma posição para a direita. Quando a comparação deixa de ser verdadeira, a posição de inserção foi encontrada. Dessa forma, a parte inicial da fila cresce gradualmente até que todo o array esteja ordenado.

Seu custo, no pior caso, é quadrático, pois cada novo elemento pode precisar ser comparado com todos os elementos já ordenados.

Merge Sort

O `merge sort` foi implementado com três funções: `merge`, `mergeSort` e `sortByScoreMerge`.

A função `merge` recebe:

```
Player* arr, Player* aux, int start, int mid, int end
```

Ela recebe um array `arr` contendo dois fragmentos já ordenados: um entre `start` e `mid`, e outro entre `mid + 1` e `end`. O papel dessa função é juntar esses dois blocos em ordem crescente, usando `comesFirst` para comparar sempre os elementos da frente de cada fragmento. O resultado é escrito em `aux` e depois copiado de volta para `arr`.

A função `mergeSort` recebe:

```
mergeSort(Player* arr, Player* aux, int start, int end)
```

Ela ordena recursivamente o trecho do array entre `start` e `end`. O caso base ocorre quando o fragmento tem um único elemento ou quando os índices já não definem uma parte válida do vetor. Nesse ponto, o trecho já está ordenado por definição.

A partir daí, o problema é dividido em duas metades: esquerda e direita. Cada metade é ordenada separadamente por chamadas recursivas, e depois as duas partes são combinadas pela função `merge`. Esse processo segue a estratégia de divisão e conquista.

O `sortByScoreMerge` foi responsável apenas por alocar o vetor auxiliar `aux` com tamanho igual ao número de jogadores armazenados. Isso foi necessário porque o processo de intercalação precisa de um espaço temporário para reconstruir os fragmentos ordenados.

O custo do *merge sort* é $O(n \log n)$ no pior caso. A divisão do problema gera aproximadamente $\log n$ níveis de recursão, e em cada nível o custo de juntar os blocos é linear. Em troca dessa eficiência, o método exige memória adicional $O(n)$.

Como a ordenação foi aproveitada para formar grupos

A ordenação foi usada como etapa anterior à formação de grupos. Ou seja, na formação de grupos pressupomos que o array já foi ordenado. Como os jogadores ficam organizados por score crescente, com desempate por timestamp, a busca por um grupo válido deixa de exigir comparações entre combinações de jogadores e passa a examinar apenas blocos consecutivos.

Na prática, foi usada uma janela deslizante de tamanho `groupSize`. O primeiro grupo candidato é formado pelos `groupSize` primeiros jogadores ordenados. Em seguida, a janela avança uma posição por vez, sempre verificando se a diferença entre o maior e o menor score da janela é menor ou igual a `delta`. Se isso acontecer, o grupo é aceito; caso contrário, a busca continua até não haver mais janelas possíveis.

Essa estratégia é determinística e sempre retorna o primeiro grupo válido na ordem atual do array ou um ponteiro nulo em caso de não ser possível encontrar um grupo que satisfaça as exigências. Além disso, ela aproveita diretamente o fato de o array estar ordenado, evitando uma busca combinatória mais cara.

Custo computacional das principais operações do sistema

- **Inserção (`insert`):** $O(1)$

A inserção é constante porque o novo jogador é colocado sempre na posição `size`, isto é, no final do array. Não há realocação nem deslocamento de elementos, desde que ainda haja capacidade disponível.

- **Remoção (`removePlayer`):** $O(n)$

A remoção exige busca linear pelo ID do jogador. Depois que o elemento é encontrado, os jogadores à direita são deslocados uma posição para a esquerda. No pior caso, isso pode atingir todos os elementos da fila.

- **Ordenação com Insertion Sort (`sortByScoreInsertion`):** $O(n^2)$

O algoritmo tem dois níveis de iteração: um para escolher o elemento atual e outro para deslocar os elementos maiores já ordenados. Quando a fila está em ordem desfavorável, o número de comparações e deslocamentos cresce quadraticamente.

- **Ordenação com Merge Sort (`sortByScoreMerge`):** $O(n \log n)$

O problema é dividido em metades sucessivas até fragmentos de tamanho 1, e cada nível da recursão realiza intercalações lineares. O resultado é um custo assintoticamente melhor do que o insertion sort, embora com uso adicional de memória $O(n)$.

- **Formação de grupo (`formGroup`):** $O(n)$

A verificação usa uma janela deslizante sobre o vetor ordenado. Para cada posição, apenas o bloco consecutivo de tamanho `groupSize` precisa ser analisado. Se um grupo é encontrado, a remoção dos jogadores escolhidos também pode exigir deslocamento linear, mantendo o custo total linear no pior caso.

- **Recuperação dos jogadores (`getWaitingPlayers`):** $O(n)$

O método cria um novo array e copia todos os jogadores atualmente armazenados. Como cada elemento precisa ser visitado uma vez, o custo é linear no número de jogadores na fila.

- **Reset do Matchmaking** (`getWaitingPlayers()`): $O(1)$

O método simplesmente atribui o valor 0 ao atributo `size` do `Matchmaking` em questão. Portanto, custo constante.

Teste de desempenho

Além dos testes funcionais, foi feita uma comparação entre os algoritmos de ordenação com entradas de tamanhos crescentes. Para garantir uma comparação justa, os mesmos jogadores foram usados em ambos os testes de ordenação dentro de cada repetição. A geração dos dados foi feita com `mt19937` e semente fixa, o que mantém a repetibilidade dos resultados.

Cada tamanho de entrada foi testado 5 vezes, e o tempo médio foi calculado com `chrono` em microssegundos. Os tamanhos utilizados foram: 10, 100, 1000, 5000, 10000, 50000 e 100000 jogadores.

A tabela abaixo mostra os resultados obtidos em microsegundos:

Tamanho da entrada (n)	Tempo médio MS - $O(n \log n)$	Tempo médio IS - $O(n^2)$
10	$3\mu s$	$1\mu s$
100	$37\mu s$	$70\mu s$
1000	$560\mu s$	$7234\mu s$
5000	$3364\mu s$	$154025\mu s$
10000	$7856\mu s$	$722895\mu s$
50000	$38611\mu s$	$16702207\mu s$
100000	$101672\mu s$	$68601504\mu s$

Os resultados estão de acordo com a complexidade esperada. Para entradas pequenas, os tempos são próximos, e diferenças pequenas podem ser influenciadas por overhead de execução, alocação e medição. No entanto, conforme o tamanho cresce, o comportamento dos algoritmos se distancia bastante.

O *insertion sort* cresce rapidamente porque cada novo elemento pode ser comparado com vários elementos já ordenados. Isso explica o salto para dezenas de milhões de microssegundos nas maiores entradas. Já o *merge sort* apresenta crescimento bem mais suave, mantendo tempo muito menor em todas as entradas médias e grandes. Em $n = 100000$, por exemplo, o *merge sort* executou em $101672 \mu s$, enquanto o *insertion sort* chegou a $68601504 \mu s$.

Essa comparação experimental confirma a análise teórica: o *merge sort* é a melhor escolha para cenários com muitos jogadores, enquanto o *insertion sort* é aceitável apenas em entradas pequenas ou em situações em que o vetor já esteja quase ordenado.